

Uart_Esp32 - Projektdokumentation - Firmware

Projektname	Uart_Esp32
Dokumentenart	Projektdokumentation
Thema	Firmware für ESP32-WROOM-32 / NodeMCU-32S
Erstellungsdatum	2026-09-17
Autor / verantwortliche Person	OpenAI ChatGPT (technische Erstellung)
Version	v0.2
Status	Entwurf
Änderungsdatum	2026-09-17
Ablageort	Projektordner Uart_Esp32/docs
Referenzen	Regeln_Projektdokumentation_PDF_DOCX_Pflicht_(5).pdf; vorheriger Projektstand Wasser_ESP32_WROOM32_Lite

Änderungshistorie

Version	Datum	Bearbeiter	Status	Änderung
v0.1	2026-09-17	OpenAI ChatGPT	Entwurf	Reduzierter WROOM32-Projektstand erstellt
v0.2	2026-09-17	OpenAI ChatGPT	Entwurf	Projekt in Uart_Esp32 umbenannt und Dokumentationsregeln umgesetzt

Dokumentations- und Auslieferungsregeln

Diese Auslieferung orientiert sich an `Regeln_Projektdokumentation_PDF_DOCX_Pflicht_(5).pdf` (Stand 2026-06-18). Für die Projektdokumentation werden PDF und DOCX mit identischem Inhalt und identischem Versionsstand bereitgestellt. Dokumenttitel und Dokumentdateinamen enthalten den Projektnamen, die Dokumentenart, das Thema und die Version; die Kennzeichnungsblöcke sind mit - getrennt.

Für das Codepaket werden eine vollständige ZIP-Datei, eine ZIP-Datei nur mit geänderten bzw. neu benötigten Dateien in korrekter Projektstruktur und eine englische Commit-Nachricht mit Datum, BL_- Titel, Zusammenfassung und Dateiliste bereitgestellt.

Bewusste Abweichung aufgrund des Auftrags: Die Dokumentationsregel fuer normale Codeänderungen sieht vor, den bestehenden Root-Ordner eines ZIP-Pakets nicht umzubenennen. In diesem Auftrag ist die Umbenennung des Projekts und des Projektverzeichnisses auf `Uart_Esp32` ausdrücklich gefordert. Deshalb wird der Root-Ordner einmalig auf `Uart_Esp32` umgestellt. Ab Version v0.2 ist `Uart_Esp32` der verbindliche Projekt-Root fuer weitere Änderungen.

Projektstruktur

```
Uart_Esp32/
platformio.ini
src/
  BatteryMonitor.cpp
  BatteryMonitor.h
  DeepSleepManager.cpp
  DeepSleepManager.h
  MqttManager.cpp
  MqttManager.h
  OtaManager.cpp
  OtaManager.h
  main.cpp
docs/
  2026-09-17 - Uart_Esp32 - Projektdokumentation - Firmware - v0.2.md
  2026-09-17 - Uart_Esp32 - Projektdokumentation - Firmware - v0.2.docx
  2026-09-17 - Uart_Esp32 - Projektdokumentation - Firmware - v0.2.pdf
  2026-09-17 - Uart_Esp32 - Commit-Nachricht - Projektumbenennung - v0.2.txt
```

Reduzierte Firmware für einen klassischen **ESP32-WROOM-32 / NodeMCU-32S**.

Das Projekt enthält nur die Infrastruktur-Funktionen aus der gewünschten Auswahl:

- WLAN-Client mit Reconnect
- Fallback Access Point
- NTP + deutsche CET/CEST-Zeitzone mit automatischer Sommer-/Winterzeit
- Webinterface
- MQTT mit Statuswerten, Fernsteuerung und Heartbeat
- Deep Sleep
 - Intervall
 - volle Stunde + Minutenoffset

- feste Uhrzeiten
- Mixed-Modus
- Einmal-Wakeup
- Nachtmodus
- batterieabhängige Intervalle
- MQTT-abhängiges Einschlafen
- Browser OTA
- ArduinoOTA / PlatformIO OTA
- HTTP Pull OTA
- NVS / Preferences
- interne ESP32-Temperatur
- Web-Reboot

Entfernt wurden u. a. VL53L1X, UART-Entfernungssensor, INA226, Regentonnenkontakt, Feeder, USB/GPIO-Portsteuerung und die komplette Distanz-Messautomatik.

Die kleine ADC-Batteriemessung ist **nur als notwendige Datenquelle für den batterieabhängigen Deep Sleep** enthalten.

Hardware / PlatformIO

platformio.ini verwendet:

```
board = nodemcu-32s
framework = arduino
```

Standardumgebung:

```
[env:uart_esp32_usb]
upload_protocol = esptool
```

Nach dem ersten USB-Flash kann optional die OTA-Umgebung benutzt werden:

```
[env:uart_esp32_ota]
upload_protocol = espota
```

upload_port und --auth in platformio.ini müssen dann zu deinem Gerät passen.

Erster Start

Auf einem frischen ESP32 sind noch keine WLAN-Zugangsdaten gespeichert. Deshalb startet der Fallback-AP:

- SSID: Uart_Esp32-Setup
- IP: 192.168.4.1
- Passwort: keines (offener AP)

Im Browser <http://192.168.4.1/> öffnen und unter **WLAN / NTP** die WLAN-Daten eintragen.

Eine leere statische IP bedeutet DHCP.

Batterie / ADC

Standard:

- ADC: GPIO34
- R1: 100 kOhm, Batterie+ -> ADC
- R2: 33 kOhm, ADC -> GND
- Mittelung: 20 Messungen
- Kalibrierfaktor: 1.000

Es werden nur **ADC1-Pins GPIO32..39** zugelassen. Dadurch kollidiert die Messung nicht mit aktivem WLAN wie es bei ADC2-Pins des klassischen ESP32 passieren kann.

Die Berechnung lautet:

$$U_{\text{Batterie}} = U_{\text{ADC}} * (R1 + R2) / R2 * \text{Kalibrierfaktor}$$

Vor dem Anschließen bitte sicherstellen, dass die maximale Spannung am ADC-Pin innerhalb des zulässigen ESP32-Bereichs bleibt.

Web-Routen

Route	Funktion
/	Hauptstatus
/network	WLAN, AP, Hostname, NTP
/mqtt	MQTT-Einstellungen / Verbindungstest
/battery	ADC-Batteriemessung für Deep Sleep
/deepsleep	Deep-Sleep-Konfiguration
/ota	Browser OTA, ArduinoOTA, Pull OTA
/reboot	Neustart, POST

MQTT

Die Topic-Basis wird auf einem frischen Gerät automatisch erzeugt:

Uart_Esp32_<Chip-ID>

Alle Fernsteuerbefehle liegen unter:

<Basis>/cmd/#

Allgemeine Befehle

Topic	Payload	Funktion
cmd/get	beliebig	Status sofort veröffentlichen
cmd/status/get	beliebig	Status sofort veröffentlichen
cmd/heartbeat_interval_s	Sekunden	Heartbeat-Intervall setzen
cmd/battery/measure	beliebig	Batterie sofort messen
cmd/reboot	beliebig	ESP32 neu starten
cmd/sleep/get	beliebig	Deep-Sleep-Status veröffentlichen

Deep-Sleep-Befehle

Alle folgenden Topics beginnen mit <Basis>/cmd/sleep/:

Untertopic	Beispiel	Funktion
enabled	1	Deep Sleep ein/aus
interval_min	15	Basis-Schlafintervall
awake_s	300	Online-Zeit
min_online_s	60	Mindest-Onlinezeit
ota_window_s	300	OTA-Schutzfenster
mqtt_ok_only	1	nur nach MQTT-Verbindung schlafen

Untertopic	Beispiel	Funktion
mqtt_timeout_s	120	maximale MQTT-Wartezeit
night_enabled	1	Nachtmodus
night_start_h	22	Nachtbeginn
night_end_h	6	Nachtende
night_interval_min	60	Schlafintervall nachts
battery_adaptive	1	batterieabhängige Intervalle
low_voltage_threshold_v	3.50	Low-Schwelle
low_voltage_interval_min	60	Low-Intervall
critical_voltage_threshold_v	3.30	Kritisch-Schwelle
critical_voltage_interval_min	180	Kritisch-Intervall
wakeup_mode	mixed	interval, full_hour, fixed, mixed
wakeup_offset_min	5	Minutenoffset für volle Stunde
fixed_times	06:00,12:00,18:00	feste Aufwachzeiten
fallback_interval_min	15	Fallback ohne gültige Uhrzeit
wake_once	2026-09-18 08:00:00	einmaliger Wakeup
wake_once_in_min	30	einmaliger Wakeup relativ
wake_once_clear	beliebig	einmaligen Wakeup löschen
now	1 oder sleep	sofort schlafen

Veröffentlichte Statuswerte

Unter <Basis>/status/ werden u. a. veröffentlicht:

- online
- hostname
- ip
- rssi
- uptime_s
- heartbeat_interval_s
- esp32_temp_c
- time
- mqtt_status
- firmware

Unter <Basis>/battery/:

- enabled
- status
- valid
- voltage_v
- adc_voltage_v
- raw_mv
- time

Unter <Basis>/sleep/:

- enabled

- wakeup_mode
- remaining_online_s
- next_wakeup_s
- next_wakeup_epoch
- next_wakeup_text
- settings_json
- beim Einschlafen zusätzlich state und reason

MQTT verwendet einen retained Last-Will auf <Basis>/status/online = false.

OTA

Browser OTA

Webseite /ota, Firmware-.bin auswählen und hochladen.

ArduinoOTA / PlatformIO OTA

Im Webinterface aktivieren und Passwort setzen. Danach uart_esp32_ota in PlatformIO verwenden. upload_port und --auth müssen angepasst werden.

HTTP Pull OTA

Im OTA-Webinterface kann eine version.json URL eingetragen werden. Erwartetes Format:

```
{
  "version": "1.2.0",
  "url": "http://server/firmware.bin"
}
```

Die reduzierte Version vergleicht Versionsnummern numerisch segmentweise, statt nur auf Ungleichheit zu prüfen.

NVS-Kompatibilität

Die wichtigen NVS-Keys der ursprünglichen Firmware wurden soweit sinnvoll beibehalten (netconf), insbesondere für:

- WLAN/AP/NTP
- MQTT
- OTA
- Deep Sleep
- Batterie-ADC

Dadurch bleiben vorhandene Einstellungen bei einem Firmwarewechsel auf demselben ESP32 in vielen Fällen nutzbar.

Prüfung

Die Quelldateien wurden mit einem lokalen C++-Syntaxcheck gegen ESP32/Arduino-API-Stubs geprüft. Eine echte PlatformIO-Kompilierung war in der Erstellungsumgebung nicht möglich, weil PlatformIO

dort nicht installiert war und kein Paketdownload verfügbar war. Vor dem Flashen daher einmal lokal ausführen:

```
pio run -e uart_esp32_usb
```

und anschließend z. B.:

```
pio run -e uart_esp32_usb -t upload
```